SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

Access control, secrets and service principals

Section 7. Least privilege, and the places a secret still leaks.

SECTION 7 of 10 WEIGHT 10% TOPIC Access control

admin@sthithapragna.com

Unity Catalog governs **data**. Workspace ACLs govern **objects**. Two systems, and a question is always about one or the other.

What everyone already has

- **Workspace admins have CAN MANAGE on every object** in the workspace. That cannot be lowered.
- **Users automatically have CAN MANAGE on objects they create.**
- Everything in the **Shared folder** is CAN MANAGE for all users. Moving something there grants it to everyone.
- So least privilege here is mostly about what you add, and about where you put things.

> ON THE EXAM
>
> Moving a notebook into Shared is a permission change, even though it looks like filing. That is worth knowing before somebody tidies up.

Inheritance, and a small information leak

- Objects in a folder **inherit all permission settings of that folder**. Managing by folder is the practical approach.
- Grant someone view on a notebook inside a folder they have no permissions on, and they can still **see the folder's name**.
- They cannot view or access anything else in it. Only the name leaks.
- So a folder named after a confidential project tells its own story to anyone with access to one file inside it.

> ON THE EXAM
>
> A minor leak with a real consequence. Name folders as though people who cannot open them will read the name.

Five levels, and one that is unusual

LEVEL	WHAT IT ALLOWS
CAN VIEW	See job details, settings and results.
CAN MANAGE RUN	Also view the Spark UI and logs, run now, run with parameters, and cancel.
IS OWNER	Also edit settings, delete the job, and modify permissions.
CAN MANAGE	The same abilities as owner, but it is a permission rather than ownership.

ON THE EXAM

A job cannot have more than one owner, and a group can never be the owner. Ownership is always exactly one identity.

Run Now uses the owner's permissions

- **A job triggered through Run Now assumes the permissions of the job owner**, not of the person who pressed it.
- So granting CAN MANAGE RUN lets somebody execute work **with the owner's access**, which may be far broader than their own.
- That is exactly why production jobs should be owned by a **service principal** with only the access the job needs.
- A job owned by a senior engineer runs with a senior engineer's access, every time anybody triggers it.

> ON THE EXAM
>
> This single fact is the strongest practical argument for service principal ownership, and it is easy to miss.

Two things they do not cover

- **Notebook workflows** that run other notebooks use the **notebook's own permissions**, not the job's.
- **Jobs submitted by API** use the notebook's default permissions unless you set an access control list in the request.
- So an externally orchestrated job can end up with permissions nobody reviewed.
- Setting the access control list explicitly at submission is the fix.

> ON THE EXAM
>
> Job permissions apply to jobs in the Jobs UI and their runs. Work reaching the platform another way follows different rules.

Four levels, and what attaching gets you

- **CAN ATTACH TO** lets somebody attach a notebook, view the Spark UI and see compute metrics.
- **CAN RESTART** adds terminating, starting and restarting.
- **CAN MANAGE** adds editing, resizing, attaching libraries and changing permissions.
- Note that attaching a notebook means running code **on that compute, as yourself**.

> ON THE EXAM
>
> Attach permission is not a small grant. It is the ability to execute arbitrary code on that machine with your own identity.

Secrets are not redacted from driver logs

- **Secrets are not redacted from the Spark driver's standard output and error streams.**
- So by default, driver logs are viewable **only by CAN MANAGE** on job and all-purpose compute in dedicated or standard access mode.
- A Spark property can relax that to let CAN ATTACH TO and CAN RESTART read them. Think before setting it.
- It is a **compute-level** property, so it must be set on each compute individually.

> ON THE EXAM
>
> The redaction people rely on happens in notebook output, not in driver logs. Printing a secret is safe; a library logging it is not.

Legacy no-isolation shared compute

- On legacy **no isolation shared** access mode, users with **CAN ATTACH TO can view service account keys** in the log file.
- The documentation says to use caution when granting that permission on such compute.
- Standard and dedicated access modes exist precisely to remove this class of problem.
- If a scenario mentions no isolation shared, treat every permission on it as broader than it looks.

> ON THE EXAM
>
> This is a legacy mode. Meeting it in a question is usually a signal that the answer involves moving off it.

Three levels, each containing the last

LEVEL	WHAT IT ALLOWS
READ	Read the secret scope, and list which secrets exist in it.
WRITE	Also write secrets to the scope.
MANAGE	Also modify the permissions on the scope.

ON THE EXAM

Access control is at the scope level, not per secret. If two secrets need different audiences, they need different scopes.

Four things about scopes

- The **creator of a scope automatically gets MANAGE**, and workspace admins have MANAGE on all scopes.
- Secret access control requires the **Premium plan or above**. Below that you grant MANAGE to all users.
- **Deleting a scope deletes every secret and every ACL in it.**
- Writing an ACL for a principal that already has one **overwrites** the existing level rather than adding to it.

> ON THE EXAM
>
> Principals are named differently by type: a user by email, a service principal by its application ID, a group by its name.

Whose permission applies

- When a secret is read from a notebook, **the permission of the user executing the command** is applied.
- They must have at least **READ** on the scope. The notebook's author is irrelevant.
- There is **no API to read the actual secret value outside a cluster**. It can only be resolved in a command.
- But remember that a job triggered by Run Now executes as the **job owner**, so it uses their scope permissions.

> ON THE EXAM
>
> Two identity rules interacting: interactive execution uses the runner, and a job run uses the owner. That combination decides several scenarios.

Why production should not run as a person

- A service principal is an identity for automation, with its own permissions and no interactive login.
- Production jobs owned by one survive the owner leaving, and run with **exactly the access the job needs**.
- Module 2 noted that production deployment mode **expects a service principal** for bundles.
- A job owned by a person is a job whose blast radius is that person's entire access.

> ON THE EXAM
>
> Least privilege on this platform is largely achieved by choosing the right identity, not by tuning permission levels.

Account group inheritance

- Workspace object permissions will soon be **inherited from all account groups a member belongs to**.
- That applies **regardless of whether those groups are assigned to the workspace**.
- So a permission granted to a group in one context may become effective somewhere it was not before.
- Databricks provides an **orphaned permissions analysis** notebook for reviewing grants ahead of the change.

> ON THE EXAM
>
> Flagged as upcoming rather than current. Worth knowing because it changes who can see what, without anybody granting anything.

What to carry into the exam

- **Unity Catalog governs data. Workspace ACLs govern objects.** Decide which system a question is about.
- **Run Now uses the job owner's permissions**, not the triggering user's.
- **Secrets are not redacted from driver logs.** That is why logs default to CAN MANAGE only.
- Secret ACLs are **per scope**, three levels, each containing the last. Reading applies the **executing user's** permission.
- Own production jobs with a **service principal**, granted only what the job needs.

> ON THE EXAM
>
> Least privilege here is achieved mostly by choosing identities and folders well, rather than by tuning individual permission levels.

> A data engineering lead owns twelve production jobs. To let the support team restart failed runs overnight, they grant the support group CAN MANAGE RUN on all twelve.
>
> A security review asks what access the support team now effectively has when they trigger a run.

What is the correct answer?

A. The support team's own access, since permissions are evaluated per user at run time.

B. The job owner's access, because a job triggered by Run Now assumes the owner's permissions.

C. No data access at all; CAN MANAGE RUN only permits starting and cancelling.

D. The access of whichever service principal the workspace runs jobs under by default.

◆ PAUSE AND DECIDE

What is the correct answer?

A The support team's own access, since permissions are evaluated per user at run time.

True for interactive execution. Not true for a job run, which assumes the owner's permissions.

B The job owner's access, because a job triggered by Run Now assumes the owner's permissions.

Stated directly in the docs. The support team can now execute work with the lead's full access.

C No data access at all; CAN MANAGE RUN only permits starting and cancelling.

Starting a run executes the job, and the job reads and writes data.

D The access of whichever service principal the workspace runs jobs under by default.

There is no such default. These jobs are owned by a person.

ON THE EXAM

The grant looks operational and is effectively a broad delegation of the owner's data access. Service principal ownership is the fix.

> An engineer reports that printing a secret in a notebook correctly shows REDACTED, so they conclude secrets cannot leak through logs. They ask for CAN ATTACH TO to be widened to the whole analytics team on a shared job cluster, so people can self-serve on debugging.
>
> A third-party library used by the job writes its connection string to standard error on failure.

What should the reviewer raise?

A Nothing; redaction covers all output paths.

B Secrets are not redacted from driver stdout and stderr, which is why driver logs default to CAN MANAGE only.

C CAN ATTACH TO does not permit log access under any configuration.

D The library should be removed, which resolves the issue entirely.

◆ PAUSE AND DECIDE

What should the reviewer raise?

A Nothing; redaction covers all output paths.

Redaction applies to notebook output. The docs state secrets are not redacted from driver streams.

B Secrets are not redacted from driver stdout and stderr, which is why driver logs default to CAN MANAGE only.

The engineer's observation is correct and their conclusion does not follow. The library's stderr write is exactly the exposed path.

C CAN ATTACH TO does not permit log access under any configuration.

A Spark property exists to allow precisely that, which is the risk being requested.

D The library should be removed, which resolves the issue entirely.

Addresses one library. The default log restriction protects against every future one.

ON THE EXAM

The redaction the engineer observed is real. It just protects a different path from the one their library uses.

> A platform team creates one secret scope holding credentials for eight different source systems. They now need the finance team to access only the two finance credentials, and not the other six.
>
> They ask how to grant READ on individual secrets within the scope.

What should they be told?

A Grant READ on the two secrets individually using the secrets CLI.

B Secret access control is at scope level. Split the credentials into separate scopes.

C Use a Unity Catalog row filter on the secret scope.

D Grant the finance team WRITE, which restricts them to their own secrets.

◆ PAUSE AND DECIDE

What should they be told?

A Grant READ on the two secrets individually using the secrets CLI.

Access control is applied to the scope, not to individual secrets.

B Secret access control is at scope level. Split the credentials into separate scopes.

The scope is the unit of access control, so different audiences require different scopes.

C Use a Unity Catalog row filter on the secret scope.

Row filters govern table data. A secret scope is a workspace object, not a table.

D Grant the finance team WRITE, which restricts them to their own secrets.

WRITE is broader than READ, not narrower. Each level contains the previous one.

ON THE EXAM

C is worth noticing as the two-systems confusion in action: a Unity Catalog feature applied to a workspace object.

> A tidy-up moves a set of analysis notebooks into the Shared folder so the team can find them. One notebook contains an embedded query against a sensitive HR table, though it holds no data itself.
>
> Nobody changed any permissions.

What has changed?

A. Nothing. Moving a notebook does not alter permissions.

B. All workspace users now have CAN MANAGE on those notebooks, though Unity Catalog still governs the HR data.

C. All workspace users can now read the HR table through the notebook.

D. The notebooks inherit the permissions of their previous folder, which travel with them.

◆ PAUSE AND DECIDE

What has changed?

A Nothing. Moving a notebook does not alter permissions.

Objects inherit the permissions of the folder they are in, and Shared is CAN MANAGE for all users.

B All workspace users now have CAN MANAGE on those notebooks, though Unity Catalog still governs the HR data.

Both halves matter: the object permission changed, and the data permission did not.

C All workspace users can now read the HR table through the notebook.

Running the notebook applies the runner's own Unity Catalog permissions, which they do not have.

D The notebooks inherit the permissions of their previous folder, which travel with them.

Inheritance follows the current location, not the previous one.

ON THE EXAM

C is the important wrong answer. The two systems are independent, so exposing the notebook does not expose the data.

> An external orchestrator submits jobs to Databricks through the API. A review finds that these runs have broader access than the equivalent jobs defined in the workspace, which are carefully permissioned.
>
> The submission request does not include an access control list.

What explains this?

A. API-submitted jobs use the notebook's default permissions unless an access control list is set in the request.

B. API-submitted jobs always run as a workspace admin.

C. The orchestrator's service principal has been granted CAN MANAGE on the workspace.

D. Job ACLs do not exist for jobs created outside the Jobs UI, so no permissions apply at all.

◆ PAUSE AND DECIDE

What explains this?

A API-submitted jobs use the notebook's default permissions unless an access control list is set in the request.

Stated in the docs as an exception to job access control. Setting the list explicitly at submission is the fix.

B API-submitted jobs always run as a workspace admin.

Invented, and it would be a far larger problem than described.

C The orchestrator's service principal has been granted CAN MANAGE on the workspace.

Possible in general, and the missing access control list is the documented explanation that fits.

D Job ACLs do not exist for jobs created outside the Jobs UI, so no permissions apply at all.

Permissions still apply; they are simply the notebook's defaults rather than a reviewed set.

ON THE EXAM

The last line of the scenario is the answer. When a question tells you a field was omitted, that omission is usually the mechanism.

> A production pipeline is owned by an engineer who is leaving the company. The team plans to transfer ownership to their group, so that the whole team collectively owns it.
>
> They also want the pipeline to keep only the access it currently needs.

What should they do?

A. Assign IS OWNER to the group, which distributes ownership across the team.

B. Transfer ownership to a service principal granted only the access the pipeline needs, and give the group CAN MANAGE.

C. Leave ownership with the departing engineer and disable their account.

D. Assign IS OWNER to a second engineer, and add the group as a co-owner.

◆ PAUSE AND DECIDE

What should they do?

A Assign IS OWNER to the group, which distributes ownership across the team.

A group cannot be assigned IS OWNER. Ownership is always exactly one identity.

B Transfer ownership to a service principal granted only the access the pipeline needs, and give the group CAN MANAGE.

Ownership survives people leaving, access is scoped to the job, and the team still administers it.

C Leave ownership with the departing engineer and disable their account.

Leaves production owned by a disabled identity, and runs would carry that identity's permissions.

D Assign IS OWNER to a second engineer, and add the group as a co-owner.

There cannot be more than one owner, and it recreates the same problem with a different person.

ON THE EXAM

Both constraints are in the answer: one owner only, never a group. A service principal is the identity that satisfies both.